K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

Data Structures(ETCCDS202)

Assignment - 2

Assignment Title: Linear Data Structures Toolkit (LDST)

Submitted By: NEHA Submitted to : Dr. Neetu Chauhan Roll Number: 2501730310 Program: B.Tech CSE (AI & ML) Course Title : Data Structures

1. Introduction

This assignment focuses on the implementation and understanding of **Linear Data Structures**, where elements are arranged sequentially in a linear order. These structures form the foundation of efficient data storage and manipulation in computer science.

The objective of this project is to design and implement a **Linear Data Structures Toolkit (LDST)** using Python. The toolkit includes the following components:

- Dynamic Array
- Singly Linked List
- Doubly Linked List
- Stack (implemented using a linked list)
- Queue (implemented using a linked list)
- Balanced Parentheses Checker (application of stack)

This assignment provides both theoretical understanding and practical implementation of these data structures.

2. Dynamic Array

2.1 Concept

A dynamic array is a data structure that automatically resizes itself when its capacity is reached. Unlike static arrays, it allows flexible memory allocation.

2.2 Working Principle

- The array starts with an initial capacity (e.g., 2 or 4).
- When the array becomes full, a new array with double the capacity is created.
- All existing elements are copied into the new array.
- The new element is then inserted.

2.3 Amortized Time Complexity of append()

The time complexity of the append() operation is:

- **O(1)** for most insertions
- **O(n)** when resizing occurs (due to copying of elements)

However, resizing does not occur frequently. Therefore, the overall average cost per operation is:

Amortized Time Complexity = O(1)

2.4 Explanation

Although some operations take linear time during resizing, the majority of operations are constant time. Hence, the average performance remains efficient.

3. Singly Linked List

3.1 Concept

A singly linked list consists of nodes, where each node contains:

- Data
- A reference (pointer) to the next node

3.2 Operations and Time Complexity

Operation	Time Complexity
Insert at beginning	O(1)
Insert at end	O(n)
Delete by value	O(n)
Traversal	O(n)

3.3 Advantages

- Dynamic size (no fixed capacity)
- Efficient insertion and deletion (no shifting required)

4. Doubly Linked List

4.1 Concept

A doubly linked list extends the singly linked list by adding a second pointer in each node:

- Pointer to the next node
- Pointer to the previous node

4.2 Operations and Time Complexity

Operation	Time Complexity
Insert after a node	O(n)
Delete at a position	O(n)

4.3 Advantages

- Traversal is possible in both directions
- Easier deletion compared to singly linked lists

5. Stack (Using Singly Linked List)

5.1 Concept

A stack is a linear data structure that follows the **LIFO (Last In, First Out)** principle.

5.2 Operations and Time Complexity

Operation	Time Complexity
push()	O(1)
pop()	O(1)
peek()	O(1)

5.3 Implementation Detail

All operations are performed at the head of the linked list, ensuring constant time complexity.

6. Queue (Using Singly Linked List)

6.1 Concept

A queue follows the **FIFO (First In, First Out)** principle.

6.2 Operations and Time Complexity

Operation	Time Complexity
enqueue()	O(1)
dequeue()	O(1)
front()	O(1)

6.3 Implementation Detail

Both head and tail pointers are maintained to ensure efficient insertion and deletion.

7. Balanced Parentheses Checker

7.1 Concept

This application uses a stack to determine whether a given expression has balanced parentheses.

7.2 Algorithm

- Traverse the expression character by character
- Push opening brackets onto the stack
- For every closing bracket:
    - Check if it matches the top of the stack
    - If not, the expression is unbalanced
- At the end:
    - If the stack is empty → Balanced
    - Otherwise → Not Balanced

7.3 Test Cases

Input Output

([]) Balanced

([)] Not Balanced

(((Not Balanced

"" Balanced

8. Conclusion

This assignment provided a comprehensive understanding of linear data structures and their practical implementation. The key learnings include:

- Understanding dynamic resizing in arrays
- Importance of linked lists for dynamic memory management
- Efficient implementation of stack and queue using linked lists
- Application of stack in solving real-world problems

Overall, this project strengthened both conceptual knowledge and programming skills in data structures.

9. References

- GeeksforGeeks – Data Structures
- Python Documentation
- Class Lecture Notes

10. Output

DYNAMIC ARRAY

Resizing from 2 to 4

Resizing from 4 to 8

Resizing from 8 to 16

1 2 3 4 5 6 7 8 9 10

Pop: 10

Pop: 9

1 2 3 4 5 6 7 8

SINGLY LINKED LIST

20 10 30 40

20 10 40

DOUBLY LINKED LIST

1 2 5 3

1 5 3

STACK

Peek: 20

Pop: 20

QUEUE

Front: 10

Dequeue: 10

PARENTHESES CHECK

([]) → Balanced

([)] → Not Balanced

(((→ Not Balanced

"" → Balanced

1. Execution Note

The program was successfully executed using Python, and the output was generated through the terminal and stored in output.txt.